For problems 2 through 6, use the following database for a gym

ATTENDANCE

MEMBERID	CHECKINDAY
10	2024-06-10
3	2024-06-11
4	2024-06-12
2	2024-06-13
9	2024-06-14
2	2024-06-15
9	2024-06-16
9	2024-06-17
1	2024-06-18
3	2024-06-19

CLASSES

CLASSID	CLASSNAME	TRAINERID	SCHEDULE
1	Yoga	1	Mon 8AM
2	HIIT	3	Tue 6PM
3	Strength Training	4	Wed 5PM
4	Cardio	1	Thu 7AM
5	Weight Loss	2	Fri 9AM
6	Pilates	3	Sat 10AM
7	Cycling	1	Sun 8AM
8	Zumba	1	Tue 5PM
9	Boxing	4	Wed 6PM
10	CrossFit	1	Thu 4PM

CLASSSIGNUPS

MEMBERID	CLASSID
3	8
8	9
3	7
4	10
9	4
7	9
2	8
8	8
9	3
4	2

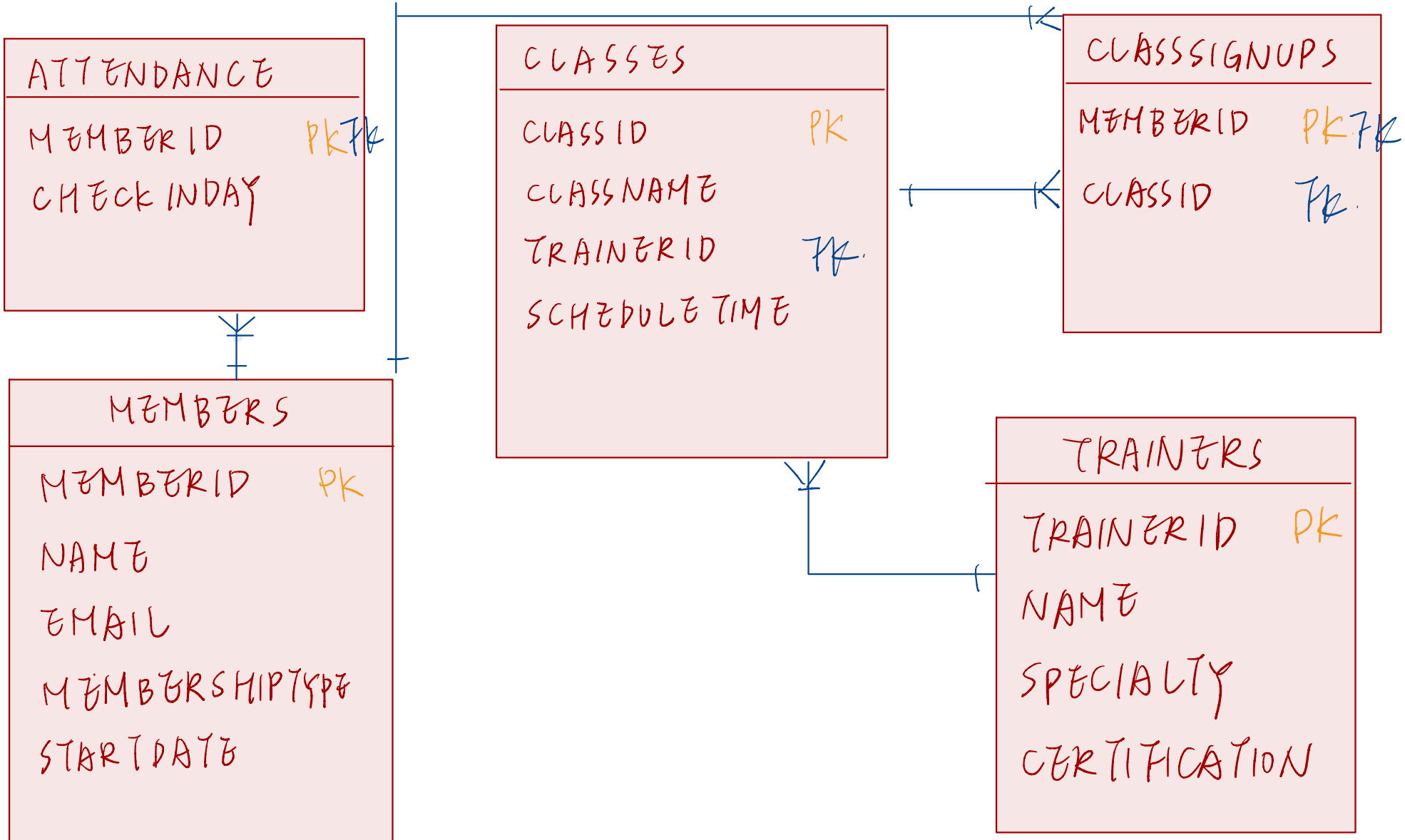
MEMBERS

MEMBERID	NAME	EMAIL	MEMBERSHIPTYPE	STARTDATE
1	Member 1	member1@example.com	Gold	2024-01-01 00:00:00.000
2	Member 2	member2@example.com	Gold	2024-01-02 00:00:00.000
3	Member 3	member3@example.com	Platinum	2024-01-03 00:00:00.000
4	Member 4	member4@example.com	Platinum	2024-01-04 00:00:00.000
5	Member 5	member5@example.com	Gold	2024-01-05 00:00:00.000
6	Member 6	member6@example.com	Silver	2024-01-06 00:00:00.000
7	Member 7	member7@example.com	Silver	2024-01-07 00:00:00.000
8	Member 8	member8@example.com	Silver	2024-01-08 00:00:00.000
9	Member 9	member9@example.com	Gold	2024-01-09 00:00:00.000
10	Member 10	member10@example.com	Gold	2024-01-10 00:00:00.000

TRAINERS

TRAINERID	NAME	SPECIALTY	CERTIFICATION
1	Trainer 1	Strength	CPT
2	Trainer 2	Yoga	RYT-500
3	Trainer 3	Cardio	CPT
4	Trainer 4	Weight Loss	NASM
5	Trainer 5	HIIT	ACE

Problem 2
Draw below the data model for this database (include attributes and identify all keys)



Problem 3
SQL Query Generation

The gym wishes to identify the most popular trainers. On the following page, write a query to find the top 3 trainers based on how many different members signed up for their classes (not how many students in total). Return the Trainer ID, Trainer Name, and total number of different students. Sort the list with the most popular trainer first.

```
SELECT T.Trainer_ID, T.Name AS Trainer_Name. , total number

FROM. TRAINERS. . T

JOIN  CLASSES. . C
 ON   TRAINERID
JOIN  CLASSSIGNUPS . C
 ON   CLASS ID.

GROUP BY TRAINER_ID
ORDER BY TotalStudents.
 LIMIT (3).
```

Problem 3 - Solution
SQL Query Generation

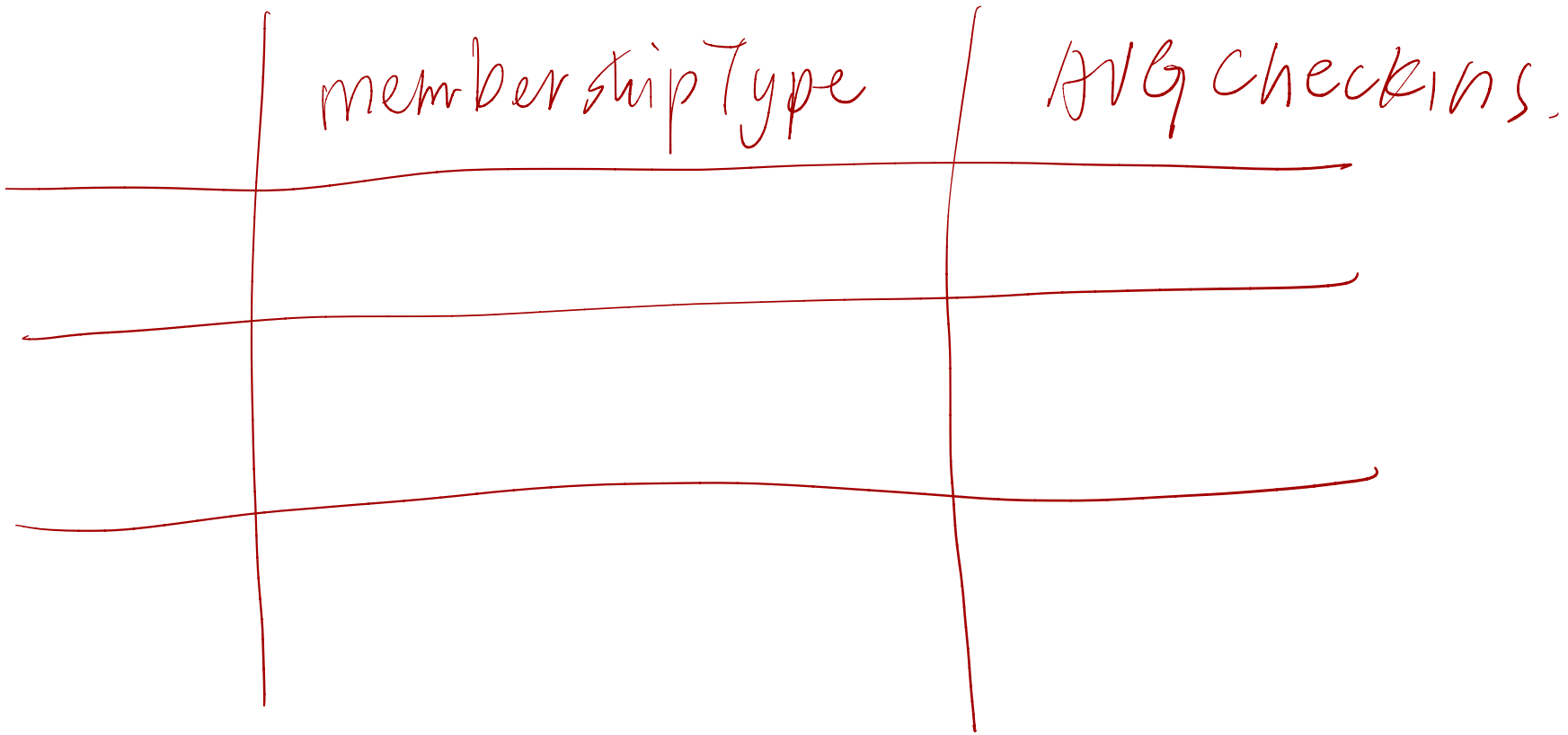


Problem 4
SQL Query Interpretation

On the following page, create the results table/report that the following query generate (in table form with the actual contents of the results):

```
SELECT m.MEMBERSHIPTYPE,
        AVG(checkin_count) AS AVGCHECKINS
FROM (
        SELECT a.MEMBERID, COUNT(a.CHECKINDAY) AS checkin_count
        FROM ATTENDANCE a GROUP BY a.MEMBERID
) checkin_data
JOIN MEMBERS m ON checkin_data.MEMBERID = m.MEMBERID
GROUP BY m.MEMBERSHIPTYPE
ORDER BY AVGCHECKINS DESC;
```

Problem 4 - Solution
SQL Query Interpretation



Problem 5
SQL Query Generation

The gym wants to reward its most active members. On the following page, write a query to rank members based on their total check-ins over the past 3 months. If two members have the same number of check-ins, assign them the same rank. Your query should return the member ID, number of check-ins, and rank.

Note: although our sample data only contains a small number of rows, assume that your query will run against a larger, multi-year version of the database

```
SELECT a.member_id, COUNT(checkindays)
    AS number of checkin

DENSE_RANK() OVER (
    PARTITION BY member ID
    ORDER BY DESC
    ) AS Rank,

FROM ATTENDANCE a
```

WHERE checkinDAY >= DATEADD(month, -3, current_date)

GROUP BY member_id.

Problem 5 - Solution
SQL Query Generation

Problem 6
SQL Query Interpretation

On the following page, create the results table/report that the following query generate (in table form with the actual contents of the results):

```
WITH CheckinLag AS (
    SELECT MEMBERID, CHECKINDAY,
           LAG(CHECKINDAY) OVER (PARTITION BY MEMBERID ORDER BY CHECKINDAY) AS PrevCheckin
    FROM ATTENDANCE
),
Streaks AS (
    SELECT MEMBERID, CHECKINDAY,
           CASE WHEN PrevCheckin = CHECKINDAY - INTERVAL '1 DAY' THEN 0 ELSE 1 END AS StreakBreak
    FROM CheckinLag
),
RunningStreaks AS (
    SELECT MEMBERID, CHECKINDAY,
           SUM(StreakBreak) OVER (PARTITION BY MEMBERID ORDER BY CHECKINDAY ROWS UNBOUNDED PRECEDING) AS StreakID
    FROM Streaks
)
SELECT MEMBERID, MAX(StreakLength) AS LongestStreak
FROM (
    SELECT MEMBERID, StreakID, COUNT(*) AS StreakLength
    FROM RunningStreaks
    GROUP BY MEMBERID, StreakID
) StreakCounts
GROUP BY MEMBERID
ORDER BY LongestStreak DESC;
```

MEMBERID	LongestStreak
9	2
2	1
3	1

Problem 6 - Solution
SQL Query Generation

Problem 6 - Solution
SQL Query Generation

Intermediate Tables (Optional – for partial credit)

CheckInLag

Streaks

RunningStreaks

* To maximize partial credit opportunity, show the results of each CTE as well as the results of the final query